

# Internet of Things (IoT) Lab

Lab 03: Direct Memory Access & Debugging

Georgios Mentzos, Miran Tobar, Jörg Henkel | June 11, 2026

CHAIR FOR EMBEDDED SYSTEMS (CES)

In this lab:

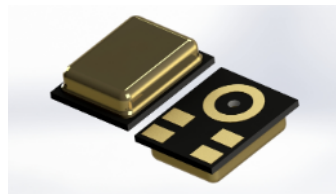
- Learn why DMA is preferable to CPU-driven data movement
- Configure USART1 transmit and receive with GPDMA1
- Understand callbacks, re-arming, and DMA-friendly buffers
- Use the on-board microphone via the BSP
- Use GDB and ST-Link to inspect and debug embedded faults

## Goal

By the end of this lab, you should be able to configure basic DMA transfers, reason about buffer ownership, and use a debugger to inspect memory, registers, and fault causes on the target.

# Microphone MP23DB01HP

- The board also contains two MP23DB01HP digital MEMS microphones. Their high-rate data makes audio signals a perfect fit for **DMA**
- In this lab, we use **Mic1** connected to the **ADF** interface
- Mic1 clock: PE9 / MIC\_CCK0, Mic1 data: PE10 / MIC\_SDINx



RHLGA 5LD (3.5 x 2.65 x 0.98 mm)

Figure: MP23DB01HP digital microphone

## Audio rate parameters

$$f_{\text{sample}} = \frac{f_{\text{mic\_clk}}}{\text{decimation}} \Rightarrow \frac{1.024 \text{ MHz}}{64} = 16 \text{ kHz}$$

- Higher sample rate means more DMA traffic

Source: STMicroelectronics, *B-U585I-IOT02A user manual* and *MP23DB01HP datasheet*

# What is DMA?

- **DMA** stands for **D**irect **M**emory **A**ccess
- Without DMA, the CPU must move each data item itself
- With DMA, a DMA controller transfers data between peripherals and memory
- This reduces CPU load and improves handling of regular data streams
- The CPU is mainly involved in setup, completion handling, and processing

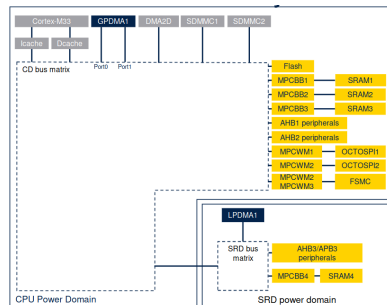


Figure: DMA BUS connection

## Typical use case

DMA is useful when data movement is regular, repetitive, or latency-sensitive, for example UART, SPI, ADC, DAC, or audio buffers.

# DMA on the STM32U5: GPDMA vs. LPDMA

- **GPDMA1**: general-purpose DMA for regular transfers (what we will use)
- **LPDMA1**: intended for low-power scenarios
- Common transfer directions:
  - peripheral-to-memory
  - memory-to-peripheral
  - memory-to-memory

## Channels, Requests, and Priorities

- A DMA controller provides multiple configurable channels. A channel is assigned to one transfer request at a time
- Transfers can be triggered via software or directly by an peripherals
- Priority decides which channel is serviced first when channels compete

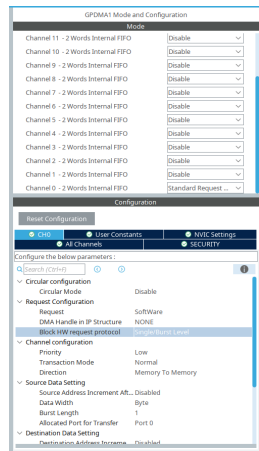


Figure: GPDMA configuration in CubeMX

- Port 0 should be typically allocated for transfers to/from peripherals
- Port 1 should be typically allocated for transfers to/from memory
- Source and destination address
- Source and destination increment mode
- Data width and transfer length
- Request source, request mode, and priority

**Read the documentation for info on how to setup DMA for your use case!**

| Aspect        | Standard Request Mode  | Linked-List Mode               |
|---------------|------------------------|--------------------------------|
| Setup         | Single transfer        | Multiple descriptors           |
| Memory layout | Contiguous buffers     | Scatter/gather possible        |
| Complexity    | Low                    | Higher                         |
| Overhead      | Minimal                | Descriptor management          |
| Typical use   | UART, SPI, ADC streams | Chained or irregular transfers |

# DMA Polling Mode

- The CPU starts the DMA transfer
- The CPU repeatedly checks whether the transfer is complete
- No interrupt is required
- Simple to understand, but wastes CPU time while waiting

## Main drawback

Polling mitigates much of the benefit of using DMA, because the CPU is still occupied with checking the transfer state.

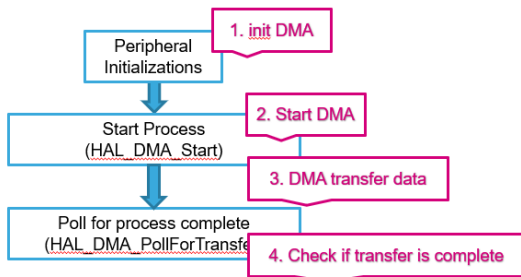


Figure: DMA transfer with polling

Source:

[https://wiki.st.com/stm32mcu/wiki/Getting\\_started\\_with\\_DMA](https://wiki.st.com/stm32mcu/wiki/Getting_started_with_DMA)

# DMA Interrupt Mode

```
/* USER CODE BEGIN 0 */  
void XferCpltCallback(DMA_HandleTypeDef *  
    hdma);  
\\...
```

```
/* USER CODE BEGIN 2 */  
hdma_memtomem_dma1_Channel1.  
    XferCpltCallback =  
        &XferCpltCallback;  
  
HAL_DMA_Start_IT(  
    &hdma_memtomem_dma1_Channel1,  
    (uint32_t)Buffer_Src,  
    (uint32_t)Buffer_Dest, 10);
```

Source: [https://wiki.st.com/stm32mcu/wiki/Getting\\_started\\_with\\_DMA](https://wiki.st.com/stm32mcu/wiki/Getting_started_with_DMA)

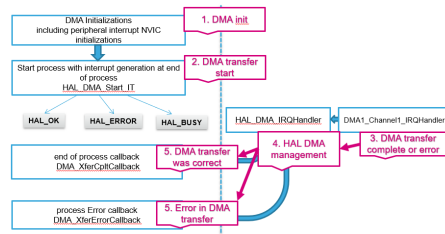


Figure: DMA transfer with completion interrupt

- The DMA copies Buffer\_Src to Buffer\_Dest
- HAL\_DMA\_Start\_IT starts the transfer with interrupt enabled



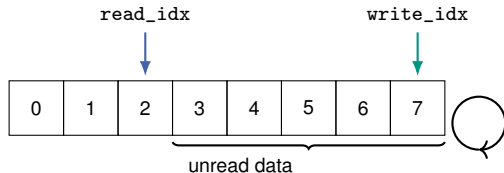
```
/* USER CODE BEGIN 4 */  
void XferCpltCallback(DMA_HandleTypeDef *hdma)  
{  
    __NOP(); // Reached only after successful transfer, set flag here  
}  
/* USER CODE END 4 */
```

## Manual callback registration

```
hdma_memtomem_dma1_Channel1.XferCpltCallback =  
    &XferCpltCallback;
```

- The callback function is stored inside the DMA handle
- The HAL DMA interrupt handler calls this function when the transfer completes
- This requires the DMA interrupt to be enabled in CubeMX/NVIC

| Structure     | Why it fits DMA                                      |
|---------------|------------------------------------------------------|
| Linear buffer | One-shot transfer into contiguous memory             |
| Double buffer | One half is processed while the other half is filled |
| Ring buffer   | Continuous stream with producer/consumer indices     |



## Ring Buffer

- Fixed-size array used as a continuous wrapping buffer
- `write_idx`: position where new data arrives
- `read_idx`: position consumed by application code

```
volatile uint32_t read_idx = 0;

void process_dma_ring(void) {
    uint32_t write_idx = RB_SIZE -
        __HAL_DMA_GET_COUNTER(&hdma_rx);

    while (read_idx != write_idx) {
        uint8_t byte = rb[read_idx];
        // Do something with byte
        read_idx++;
        if (read_idx >= RB_SIZE) {
            read_idx = 0;
        }
    }
}
```

- Embedded programs run directly on hardware without an operating system, which makes debugging a little more challenging
- `printf` debugging is useful, but limited:
  - it changes timing
  - it needs a working UART
  - it may be too late after a crash
- The solution is to use an Remote Debugging Protocol (Debugging with GDB)

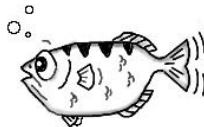


Figure: GDB Debugger Mascot

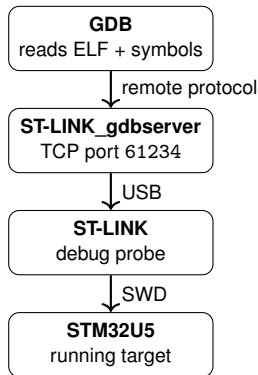
## What the debugger gives us

- stop the CPU at breakpoints
- inspect registers and memory
- watch variables while stepping
- inspect the call stack
- find where a fault happened

Source: <https://www.sourceware.org/gdb/mascot/>

# GDB + ST-LINK: What Is Happening?

- **GDB** runs on the host PC and reads the ELF file
- The ELF contains symbols, source locations, and addresses
- **ST-LINK\_gdbserver** opens a TCP port for GDB
- GDB sends debug commands to this server
- The server translates these commands into ST-LINK operations
- ST-LINK controls the STM32 over the **SWD** debug interface



Breakpoints, stepping, register reads, and memory reads all travel through this chain.

# Minimal GDB Session

## Start the debug server

```
ST-LINK_gdbserver -l 1 -d -v \  
-cp <path-to-cubeprogrammer-bin>
```

## Connect with GDB

```
arm-none-eabi-gdb build/Debug/<program>.elf
```

```
(gdb) target extended-remote :61234  
(gdb) monitor reset halt  
(gdb) load  
(gdb) break main  
(gdb) continue
```

## Useful commands

|                |                           |
|----------------|---------------------------|
| step           | <i># step into</i>        |
| next           | <i># step over</i>        |
| continue       | <i># resume execution</i> |
| break main     | <i># set breakpoint</i>   |
| print var      | <i># print variable</i>   |
| info registers | <i># CPU registers</i>    |
| x/16x addr     | <i># inspect memory</i>   |
| backtrace      | <i># call stack</i>       |
| watch var      | <i># stop on change</i>   |

## VSCode

Instead you can also debug your application via the STM32-Extension from VSCode, by simply clicking on the debug button in the footer.

1. Configure UART transmit and receive with DMA
2. Debug a faulty memory-to-memory DMA program with GDB
3. Use BSP audio input to fill a sample buffer and compute RMS level

The goal of this you to know when and how to use DMA in your projects. And also be able to use the debugger to help you fix bugs, both memory and non-memory related.

# Task 1: UART DMA Setup

In this task, configure and use DMA to transmit some String from a Buffer directly to the USART1 peripheral, circumventing the CPU.

- Create a new CubeMX project for B-U585I-I0T02A and enable USART1 with 115200, 8N1
- Enable GPDMA1 by adding two DMA channels for USART1\_RX and USART1\_TX
- Enable the USART1 global interrupt and the DMA NVIC channel interrupts
- Configure USART1\_TX channel for **memory-to-peripheral** transfer
  - source: memory buffer
  - destination: UART transmit data register
  - make sure to enable increment src after transfer
- Configure USART1\_RX channel for **peripheral-to-memory** transfer
  - source: UART receive data register
  - destination: memory buffer
  - enable circular mode for Port 0
- Use **Standard Request Mode** and 1 Byte transfers for both

# Task 1: Transmit and Receive Buffers

Now before you continue take a couple of minutes and make sure you understand what the various config options you just selected do. Then open the generated code and start with the implementation: Define fixed-size buffers in `main.c`.

```
uint8_t pRxBuff[10];  
uint8_t pTxBuff[10] = "Count:  \r\n";  
uint8_t u8Inc = '0';
```

- Start reception once after initialization
- Transmit the counter buffer once per second
- Observe that the CPU does not copy each byte into the UART manually
- The HAL provides its own interface for combining UART and DMA, which allows us to skip manually registering callbacks.

```
HAL_UART_Receive_DMA(&huart1, pRxBuff, 10);
```



# Task 1: Main Loop

Now we want you send a short UART message once per second using DMA. Reception should remain active in the background. We use interrupts and callbacks instead of repeatedly polling UART status flags in the main loop.

Your main loop should:

- update one character in `pTxBuff` so that the transmitted message contains a visible counter,
- increment the counter from '0' to '9',
- wrap the counter back to '0' after '9',
- start the transmission with `HAL_UART_Transmit_DMA`,
- wait roughly one second before sending the next message.

Do not wait for UART completion by polling in the main loop.

# Task 1: Finishing up: Callbacks

In the main loop, UART transmission and reception are started with DMA and the HAL calls a callback when a DMA transfer completes. For this task, implement:

```
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart);  
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
```

- The transmit callback should indicate that a UART message was sent ( e.g. by toggling the green LED).
- The receive callback should indicate that a message was received (e.g. by toggling the red LED). It should also print something back to UART. If the channel was configured correctly, you wont have to restart the receive operation.

## Note on registering callbacks

The functions above are the legacy/default HAL callback path. Your definition overrides the weak default implementations provided by HAL. Therefore, we do not have to explicitly register callbacks here.

## Task 2: Find the Fault

In this task, you will debug a faulty DMA program with GDB.

A misconfigured DMA example is provided on ILIAS. Compile it, flash it to your board, run it, and use the debugger to identify what goes wrong.

Your task is to:

1. flash and run the provided program,
2. reproduce the crash and observe the UART output,
3. halt the program in the debugger after the fault,
4. inspect the call stack and the faulting instruction,
5. inspect the relevant source and destination memory,
6. identify the wrong DMA configuration or the wrong interpretation of copied data,
7. fix the bug and verify that the output is stable.

## Task 3: Audio Buffer with BSP

In this final task, you will use a realistic example where DMA is useful in an embedded system. The board contains digital MEMS microphones. Their data rate is high enough that reading every audio sample manually in software would be inefficient. Therefore, the BSP audio input driver already uses DMA internally.

Your task is to use the BSP audio input driver to capture microphone samples into a memory buffer and compute the RMS level of a completed audio frame.

Start by setting up the project:

- Create and configure the CubeMX project as usual.
- Add the required BSP files for audio input, following the same procedure as in Lab 02.
- Find out which MCU peripheral is connected to the digital microphone.
- Find out which pins are used for the microphone clock and data signals.
- Explain why DMA is useful for this peripheral.

## Task 3: Using the BSP with MP23DB01HP

The BSP hides most of the low-level microphone setup. Instead of manually configuring the MP23DB01HP microphone, the ADF/MDF peripheral, DMA, GPIO pins, and audio format, we describe the desired audio input stream with a configuration structure.

```
BSP_AUDIO_Init_t audio_init = {  
    .Device = AUDIO_IN_DEVICE_DIGITAL_MIC1,  
    .SampleRate = AUDIO_FREQUENCY_16K,  
    .BitsPerSample = AUDIO_RESOLUTION_16B,  
    .ChannelsNbr = 1,  
    .Volume = 50,  
};
```

This configuration selects the first digital microphone, records one mono channel, and stores each audio sample as a 16-bit value. We use a sample rate of  $f_s = 16$  kHz. Therefore, the highest representable audio frequency is approximately

## Task 3: Starting Audio Capture

Use the BSP audio input functions to initialize the peripheral and start recording:

```
BSP_AUDIO_IN_Init(OU, &audio_init);  
  
BSP_AUDIO_IN_Record(OU,  
                    (uint8_t *)mic_buffer,  
                    sizeof(mic_buffer));
```

Check the return value of both functions. If recording fails, the peripheral, BSP files, clock configuration, or DMA configuration is probably incomplete.

The audio buffer is filled continuously by DMA. The BSP uses a half-buffer workflow:

- while DMA writes one half of the buffer, the CPU can process the other half,
- the half-transfer callback indicates that the first half is ready,
- the transfer-complete callback indicates that the second half is ready.

## Task 3: Buffer Size and Processing Time

Assume the full circular DMA buffer contains  $N$  samples and the sample rate is  $f_s$ . Then the duration of a buffer with  $N/2$  samples, is  $T_{\text{half}}$  seconds of audio:

$$T_{\text{half}} = \frac{N}{2f_s}$$

After a half-transfer or transfer-complete callback, the CPU must finish processing the completed half before DMA starts writing to that same half again. For a CPU clock frequency of  $f_{\text{CPU}}$ , the available processing time is therefore approximately

$$C_{\text{process}} = f_{\text{CPU}} \cdot T_{\text{process}} = \frac{f_{\text{CPU}} N}{2f_s}$$

## Task 3: RMS over an Audio Frame

After sampling, compute the root mean square (RMS) level of one completed buffer region. For samples  $x_0, x_1, \dots, x_{N-1}$ , the RMS value is

$$x_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} x_i^2}.$$

For a more robust audio level estimate, first remove any DC offset by subtracting the mean value. This avoids measuring a constant offset as part of the audio signal level.:

$$\bar{x} = \frac{1}{N} \sum_{i=0}^{N-1} x_i, \quad x_{\text{RMS,centered}} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} (x_i - \bar{x})^2}.$$



## Task 3: Expected Result

Your program should continuously capture audio into the DMA buffer and process completed half-buffers.

For each completed half-buffer:

- select the half-buffer indicated by the BSP callback,
- compute the RMS or centered RMS value,
- print the result over UART,
- optionally display a simple text bar showing the microphone level.

The important idea is that the CPU does not read the microphone sample by sample. DMA moves the audio data into memory automatically, and the CPU only processes blocks of samples after a callback tells it that a block is ready.

## Bonus Task 4: Python Audio Capture Harness

Write a small Python tool that controls the board over UART, captures microphone data, and stores it as a playable WAV file.

Firmware side:

- Use the BSP audio input driver.
- Capture microphone samples on the STM32.
- Send the raw 16-bit mono PCM samples over UART.

Host side:

- Use `pyserial` for UART communication.
- Use Python's built-in `wave` module to write the WAV file.

Background on the WAV format: <https://docs.fileformat.com/audio/wav/>

## Bonus Task 4: BSP DMA Setup

Generate the project as before and add the BSP drivers. This time you will have to manually adjust the BSP driver.

The BSP creates the MIC1 DMA queue MdfQueue1 inside MDF\_BlockMspInit(). This queue is attached to `haudio_mdf[0]` and uses `GPDMA1_Channel16`. By default it is configured for continuous (circular) capture:

```
/* Set queue in circular mode */
if (HAL_DMAEx_List_SetCircularMode(&MdfQueue1) != HAL_OK)
{
    BSP_AUDIO_IN_Error_CallBack(0);
}
haudio_mdf[0].InitLinkedList.LinkedListMode =
    DMA_LINKEDLIST_CIRCULAR;
```

For a finite recording, you need to change this to `DMA_LINKEDLIST_NORMAL` (see next slide).

## Bonus Task 4: Finite Capture Mode

Switch the DMA queue to normal linked-list mode:

```
/* Keep the queue linear for finite captures. */  
haudio_mdf[0].InitLinkedList.LinkedListMode =  
    DMA_LINKEDLIST_NORMAL;
```

Effect:

- HAL\_DMAEx\_List\_SetCircularMode(&MdfQueue1) is no longer called.
- The linked-list node is not linked back to itself and haudio\_mdf[0] runs in normal linked-list mode.
- **The DMA transfer stops after a single finite capture buffer.**

Compute the buffer size required for your chosen recording length. A good starting point is 2 s.

## Bonus Task 4: Firmware Output Protocol

Rather than streaming raw bytes, the firmware frames each capture with a short text header and footer. This lets the host know the format and how many bytes to read:

```
static void Audio_SendPcm(void)
{
    printf("BEGIN_PCM %lu sample_rate=%lu bits=%lu channels=%lu\r\n",
        (unsigned long)AUDIO_CAPTURE_BYTES,
        (unsigned long)AUDIO_CAPTURE_SAMPLE_RATE,
        (unsigned long)AUDIO_CAPTURE_BITS,
        (unsigned long)AUDIO_CAPTURE_CHANNELS);
    (void)UART_SendBuffer((const uint8_t *)mic_buffer, AUDIO_CAPTURE_BYTES);
    UART_SendString("\r\nEND_PCM\r\n");
}
```

## Bonus Task 4: Python Setup

In the past you have used serial monitors such as `minicom` or similar to interact with the UART output from the boards. For this task you will write a Python program that fulfills that role. Use `pyserial` to handle the serial communication. For a short introduction, check out:

<https://pyserial.readthedocs.io/en/latest/shortintro.html>

You are free to use the client we provide as-is — it will work. Be aware, though, that using it might result in more work for you in the long run. We recommend treating our client as a reference for the framing protocol and overall structure, and writing your own from that inspiration.

Either way, the key point is that you understand the idea behind packaging the raw output in a frame: the board wraps each capture in a short header (e.g. the one shown earlier) so your code knows how many bytes to read from the stream. Whatever client you use must respect that.

## Bonus Task 4: Python Menu

If you decide to roll your own harness your Python program should provide a small command-line menu:

```
def print_menu():  
    print()  
    print("B-U585I UART Audio Tool")  
    print("Commands:")  
    print("  r, record  record 2 seconds and save a WAV")  
    print("  i, info    show capture format reported by the board")  
    print("  h, help    show this menu")  
    print("  q, quit    close the serial port")  
    print()
```

# Questions?

Chair for Embedded Systems (CES)  
Karlsruhe Institute of Technology (KIT)